

PCCSL507
Computer Networks

ASSIGNMENT - 02

Submitted By:

Deepthi George

S5 CSE A

Roll No:23

VJCET

Question 1 – Hello Client-Server (Beginner)

Problem Statement:

Develop a TCP client-server application where:

- The client sends the message "Hello Server".***
- The server receives the message and displays it.***
- The server replies with "Hello Client".***
- The client displays the received response.***

PROGRAM

>>SERVER

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

int main()
{
    int serverSocket, clientSocket;
    struct sockaddr_in serverAddr;
    socklen_t addrSize;

    char buffer[1024];

    /* Create Socket */

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);
```

```
if(serverSocket < 0)
{
    perror("Socket Creation Failed");
    exit(1);
}

printf("Socket Created Successfully...\n");

/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);

/* Bind */

if(bind(serverSocket,
        (struct sockaddr*)&serverAddr,
        sizeof(serverAddr)) < 0)
{
    perror("Bind Failed");
    exit(1);
}

printf("Bind Successful...\n");

/* Listen */

listen(serverSocket,5);

printf("Waiting for Client...\n");
```

```
addrSize = sizeof(serverAddr);

/* Accept */

clientSocket = accept(serverSocket,
                      NULL,
                      NULL);

if(clientSocket < 0)
{
    perror("Accept Failed");
    exit(1);
}

printf("Client Connected...\n");

/* Receive */

recv(clientSocket,
     buffer,
     sizeof(buffer),
     0);

printf("Client Says : %s\n",buffer);

/* Reply */

strcpy(buffer,"Hello Client");

send(clientSocket,
     buffer,
```

```

        strlen(buffer)+1,
        0);

    printf("Reply Sent...\n");

    close(clientSocket);

    close(serverSocket);

    printf("Connection Closed...\n");

    return 0;
}

```

>>CLIENT

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8080

int main()
{
    int sockfd;
    struct sockaddr_in serverAddr;

    char buffer[1024];

    /* Create Socket */

```

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);

if(sockfd < 0)
{
    perror("Socket Creation Failed");
    exit(1);
}

printf("Socket Created Successfully...\n");

/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);

inet_pton(AF_INET,
          "127.0.0.1",
          &serverAddr.sin_addr);

/* Connect */

if(connect(sockfd,
           (struct sockaddr *)&serverAddr,
           sizeof(serverAddr)) < 0)
{
    perror("Connection Failed");
    exit(1);
}

printf("Connected to Server...\n");

/* Read Message */
```

```
printf("Enter Message : ");
fgets(buffer,sizeof(buffer),stdin);

/* Send */
send(sockfd,
      buffer,
      strlen(buffer)+1,
      0);

printf("Message Sent...\n");

/* Receive */

recv(sockfd,
      buffer,
      sizeof(buffer),
      0);

printf("\nReply from Server : %s\n",
      buffer);

/* Close */

close(sockfd);

printf("Connection Closed...\n");

return 0;
}
```

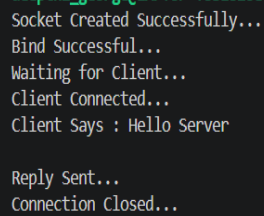
OUTPUT

>>SERVER

To compile:

gcc server.c -o server

./server



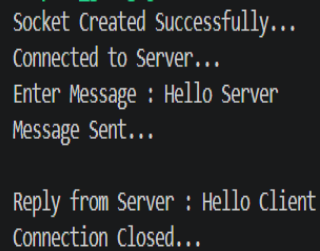
```
Socket Created Successfully...  
Bind Successful...  
Waiting for Client...  
Client Connected...  
Client Says : Hello Server  
  
Reply Sent...  
Connection Closed...
```

>>CLIENT

To compile:

gcc client.c -o client

./client



```
Socket Created Successfully...  
Connected to Server...  
Enter Message : Hello Server  
Message Sent...  
  
Reply from Server : Hello Client  
Connection Closed...
```

Question 2 – Arithmetic Server

Problem Statement:

Develop a TCP client-server application where:

- *The client inputs two integers.*
- *The client sends both numbers to the server.*
- *The server calculates:*
 - Addition
 - Subtraction

-Multiplication

-Division

PROGRAM

>>SERVER

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

int main()
{
    int serverSocket, clientSocket;
    struct sockaddr_in serverAddr;
    int numbers[2];
    int results[4];

    /* Create Socket */

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    if(serverSocket < 0)
    {
        perror("Socket Creation Failed");
        exit(1);
    }

    printf("Socket Created Successfully...\n");
```

```
/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);

/* Bind */

if(bind(serverSocket,
        (struct sockaddr*)&serverAddr,
        sizeof(serverAddr)) < 0)
{
    perror("Bind Failed");
    exit(1);
}

printf("Bind Successful...\n");

/* Listen */

listen(serverSocket, 5);

printf("Waiting for Client...\n");

/* Accept */

clientSocket = accept(serverSocket, NULL, NULL);

if(clientSocket < 0)
{
    perror("Accept Failed");
}
```

```
        exit(1);
    }

    printf("Client Connected...\n");

    /* Receive Two Integers */

    recv(clientSocket,
          numbers,
          sizeof(numbers),
          0);

    printf("Received Numbers : %d , %d\n",
          numbers[0], numbers[1]);

    /* Calculations */

    results[0] = numbers[0] + numbers[1];

    results[1] = numbers[0] - numbers[1];

    results[2] = numbers[0] * numbers[1];

    if(numbers[1] != 0)
    {
        results[3] = numbers[0] / numbers[1];
    }
    else
    {
        results[3] = 0;
    }
}
```

```

    /* Send Results */

    send(clientSocket,
          results,
          sizeof(results),
          0);

    printf("Results Sent...\n");

    /* Close */

    close(clientSocket);

    close(serverSocket);

    printf("Connection Closed...\n");

    return 0;
}

```

>>CLIENT

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

int main()
{

```

```
int sockfd;
struct sockaddr_in serverAddr;
int numbers[2];
int results[4];

/* Create Socket */

sockfd = socket(AF_INET, SOCK_STREAM, 0);

if(sockfd < 0)
{
    perror("Socket Creation Failed");
    exit(1);
}

printf("Socket Created Successfully...\n");

/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);

inet_pton(AF_INET,
          "127.0.0.1",
          &serverAddr.sin_addr);

/* Connect */

if(connect(sockfd,
          (struct sockaddr *)&serverAddr,
          sizeof(serverAddr)) < 0)
{
```

```
        perror("Connection Failed");
        exit(1);
    }

    printf("Connected to Server...\n");

    /* Input */

    printf("Enter First Integer : ");
    scanf("%d", &numbers[0]);

    printf("Enter Second Integer : ");
    scanf("%d", &numbers[1]);

    /* Send Two Integers */

    send(sockfd,
        numbers,
        sizeof(numbers),
        0);

    printf("Numbers Sent...\n");

    /* Receive Results */

    recv(sockfd,
        results,
        sizeof(results),
        0);

    /* Display Results */
```

```

printf("\n----- RESULTS ----- \n");

printf("Addition      : %d\n", results[0]);

printf("Subtraction   : %d\n", results[1]);

printf("Multiplication : %d\n", results[2]);

if(numbers[1] != 0)
{
    printf("Division      : %d\n", results[3]);
}
else
{
    printf("Division      : Cannot divide by zero\n");
}

/* Close */

close(sockfd);

printf("Connection Closed...\n");

return 0;
}

```

OUTPUT

>>SERVER

```
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Numbers : 10 , 5
Results Sent...
Connection Closed...
```

>>CLIENT

```
Socket Created Successfully...
Connected to Server...
Enter First Integer : 10
Enter Second Integer : 5
Numbers Sent...

----- RESULTS -----
Addition      : 15
Subtraction   : 5
Multiplication : 50
Division      : 2
Connection Closed...
```

Question 3 – String Processing Server

Problem Statement:

Develop a TCP client-server application where:

- ***The client inputs a string.***
- ***The server performs the following operations:***
 - ***Find the string length***
 - ***Convert the string to uppercase***
 - ***Reverse the string***
- ***The server sends the processed results.***
- ***The client displays all outputs.***

PROGRAM

>>SERVER

```
#include <stdio.h>
```



```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <ctype.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

struct Result
{
    int length;
    char uppercase[1024];
    char reverse[1024];
};

int main()
{
    int serverSocket, clientSocket;
    struct sockaddr_in serverAddr;

    char buffer[1024];
    struct Result result;

    int i;

    /* Create Socket */

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    if(serverSocket < 0)
    {
```

```
        perror("Socket Creation Failed");
        exit(1);
    }

    printf("Socket Created Successfully...\n");

    /* Server Address */

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);

    /* Bind */

    if(bind(serverSocket,
            (struct sockaddr*)&serverAddr,
            sizeof(serverAddr)) < 0)
    {
        perror("Bind Failed");
        exit(1);
    }

    printf("Bind Successful...\n");

    /* Listen */

    listen(serverSocket, 5);

    printf("Waiting for Client...\n");

    /* Accept */
```

```
clientSocket = accept(serverSocket, NULL, NULL);

if(clientSocket < 0)
{
    perror("Accept Failed");
    exit(1);
}

printf("Client Connected...\n");

/* Receive String */

recv(clientSocket,
      buffer,
      sizeof(buffer),
      0);

printf("Received String : %s\n", buffer);

/* Find Length */

result.length = strlen(buffer);

/* Convert to Uppercase */

for(i = 0; i < result.length; i++)
{
    result.uppercase[i] = toupper(buffer[i]);
}

result.uppercase[result.length] = '\0';
```

```

/* Reverse String */

for(i = 0; i < result.length; i++)
{
    result.reverse[i] = buffer[result.length - 1 - i];
}

result.reverse[result.length] = '\0';

/* Send All Results Together */

send(clientSocket,
    &result,
    sizeof(result),
    0);

printf("Processed Results Sent...\n");

/* Close */

close(clientSocket);

close(serverSocket);

printf("Connection Closed...\n");

return 0;
}

```

>>CLIENT

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

struct Result
{
    int length;
    char uppercase[1024];
    char reverse[1024];
};

int main()
{
    int sockfd;
    struct sockaddr_in serverAddr;

    char buffer[1024];
    struct Result result;

    /* Create Socket */

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if(sockfd < 0)
    {
        perror("Socket Creation Failed");
        exit(1);
    }
```

```
printf("Socket Created Successfully...\n");
```

```
/* Server Address */
```

```
serverAddr.sin_family = AF_INET;
```

```
serverAddr.sin_port = htons(PORT);
```

```
inet_pton(AF_INET,
```

```
        "127.0.0.1",
```

```
        &serverAddr.sin_addr);
```

```
/* Connect */
```

```
if(connect(sockfd,
```

```
        (struct sockaddr *)&serverAddr,
```

```
        sizeof(serverAddr)) < 0)
```

```
{
```

```
    perror("Connection Failed");
```

```
    exit(1);
```

```
}
```

```
printf("Connected to Server...\n");
```

```
/* Input String */
```

```
printf("Enter String : ");
```

```
fgets(buffer, sizeof(buffer), stdin);
```

```
/* Remove newline */
```

```
buffer[strcspn(buffer, "\n")] = '\0';

/* Send String */

send(sockfd,
    buffer,
    sizeof(buffer),
    0);

printf("String Sent...\n");

/* Receive All Results */

recv(sockfd,
    &result,
    sizeof(result),
    0);

/* Display Results */

printf("\n----- RESULTS ----- \n");

printf("String Length : %d\n", result.length);

printf("Uppercase      : %s\n", result.uppercase);

printf("Reverse        : %s\n", result.reverse);

/* Close */

close(sockfd);
```

```

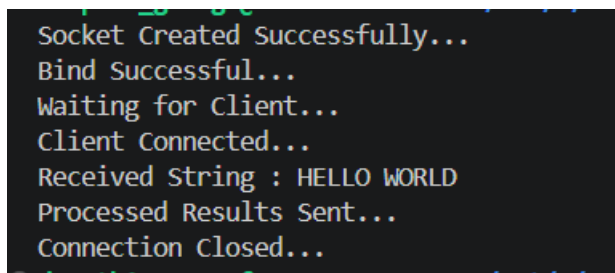
        printf("Connection Closed...\n");

        return 0;
    }

```

OUTPUT

>>SERVER

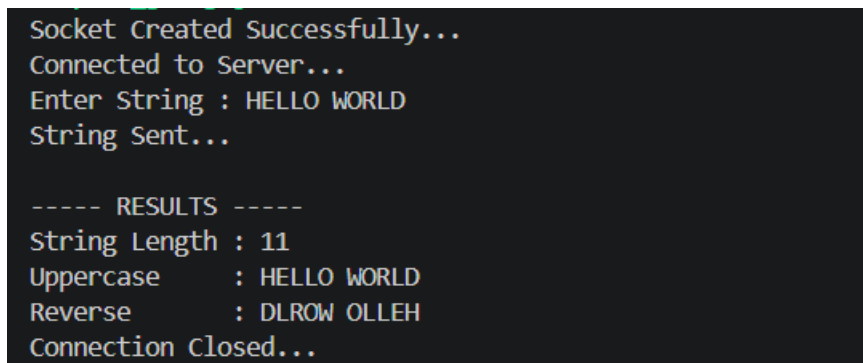


```

Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received String : HELLO WORLD
Processed Results Sent...
Connection Closed...

```

>>CLIENT



```

Socket Created Successfully...
Connected to Server...
Enter String : HELLO WORLD
String Sent...

----- RESULTS -----
String Length : 11
Uppercase      : HELLO WORLD
Reverse       : DLROW OLLEH
Connection Closed...

```

Question 4 – Array Processing Server

Problem Statement:

Develop a TCP client-server application where:

- *The client inputs N integers.*
- *The array is sent to the server.*
- *The server computes:*
 - *Sum*
 - *Average*

- *Maximum Element*
- *Minimum Element*
- *The server sends the results.*
- *The client displays them.*

PROGRAM

>>SERVER

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define MAX 100

struct Result
{
    int sum;
    float average;
    int maximum;
    int minimum;
};

int main()
{
    int serverSocket, clientSocket;
    struct sockaddr_in serverAddr;

    int n;
    int array[MAX];
    struct Result result;
```

```
int i;

/* Create Socket */

serverSocket = socket(AF_INET, SOCK_STREAM, 0);

if(serverSocket < 0)
{
    perror("Socket Creation Failed");
    exit(1);
}

printf("Socket Created Successfully...\n");

/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);

/* Bind */

if(bind(serverSocket,
        (struct sockaddr*)&serverAddr,
        sizeof(serverAddr)) < 0)
{
    perror("Bind Failed");
    exit(1);
}

printf("Bind Successful...\n");
```

```
/* Listen */

listen(serverSocket, 5);

printf("Waiting for Client...\n");

/* Accept */

clientSocket = accept(serverSocket, NULL, NULL);

if(clientSocket < 0)
{
    perror("Accept Failed");
    exit(1);
}

printf("Client Connected...\n");

/* Receive N */

recv(clientSocket,
      &n,
      sizeof(n),
      0);

/* Receive Array */

recv(clientSocket,
      array,
      sizeof(int) * n,
      0);
```

```
printf("Received Array : ");

for(i = 0; i < n; i++)
{
    printf("%d ", array[i]);
}

printf("\n");

/* Calculate Sum */

result.sum = 0;

for(i = 0; i < n; i++)
{
    result.sum = result.sum + array[i];
}

/* Calculate Average */

result.average = (float)result.sum / n;

/* Find Maximum and Minimum */

result.maximum = array[0];
result.minimum = array[0];

for(i = 1; i < n; i++)
{
    if(array[i] > result.maximum)
    {
```

```

        result.maximum = array[i];
    }

    if(array[i] < result.minimum)
    {
        result.minimum = array[i];
    }
}

/* Send Results */

send(clientSocket,
    &result,
    sizeof(result),
    0);

printf("Results Sent...\n");

/* Close */

close(clientSocket);

close(serverSocket);

printf("Connection Closed...\n");

return 0;
}

```

>>CLIENT

```

#include <stdio.h>
#include <stdlib.h>

```

```
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define MAX 100

struct Result
{
    int sum;
    float average;
    int maximum;
    int minimum;
};

int main()
{
    int sockfd;
    struct sockaddr_in serverAddr;

    int n;
    int array[MAX];

    struct Result result;

    int i;

    /* Create Socket */

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if(sockfd < 0)
```

```

{
    perror("Socket Creation Failed");
    exit(1);
}

printf("Socket Created Successfully...\n");

/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);

inet_pton(AF_INET,
          "127.0.0.1",
          &serverAddr.sin_addr);

/* Connect */

if(connect(sockfd,
          (struct sockaddr *)&serverAddr,
          sizeof(serverAddr)) < 0)
{
    perror("Connection Failed");
    exit(1);
}

printf("Connected to Server...\n");

/* Input N */

printf("Enter number of elements : ");
scanf("%d", &n);

```

```
if(n <= 0 || n > MAX)
{
    printf("Invalid number of elements.\n");
    close(sockfd);
    return 1;
}

/* Input Array */

printf("Enter %d integers:\n", n);

for(i = 0; i < n; i++)
{
    scanf("%d", &array[i]);
}

/* Send N */

send(sockfd,
    &n,
    sizeof(n),
    0);

/* Send Array */

send(sockfd,
    array,
    sizeof(int) * n,
    0);

printf("Array Sent...\n");
```



```

/* Receive Results */

recv(sockfd,
      &result,
      sizeof(result),
      0);

/* Display Results */

printf("\n----- RESULTS ----- \n");

printf("Sum      : %d\n", result.sum);

printf("Average : %.2f\n", result.average);

printf("Maximum : %d\n", result.maximum);

printf("Minimum : %d\n", result.minimum);

/* Close */

close(sockfd);

printf("Connection Closed...\n");

return 0;
}

```

OUTPUT

>>SERVER

```
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Array : 1 2 3 4 5
Results Sent...
Connection Closed...
```

>>CLIENT

```
Socket Created Successfully...
Connected to Server...
Enter number of elements : 5
Enter 5 integers:
1 2 3 4 5
Array Sent...

----- RESULTS -----
Sum      : 15
Average  : 3.00
Maximum  : 5
Minimum  : 1
Connection Closed...
```

Question 5 – Matrix Analyzer

Problem Statement:

Develop a TCP client-server application where:

- *The client inputs a square matrix of order N.*
- *The matrix is sent to the server.*
- *The server determines:*
 - *Matrix Type*
 - *Sum of Matrix Elements*
 - *Trace of Matrix*
- *The server sends the results.*
- *The client displays the received information.*

PROGRAM

>>SERVER

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define MAX 10

struct Result
{
    char type[100];
    int sum;
    int trace;
};

/* Receive exactly one integer */
int receive_int(int socket, int *value)
{
    int total = 0;
    int bytes;

    while (total < sizeof(int))
    {
        bytes = recv(socket,
                      (char *)value + total,
                      sizeof(int) - total,
                      0);

        if (bytes <= 0)
            return -1;
    }
}

```

```

        total += bytes;
    }

    return 0;
}

/* Send complete result */
int send_result(int socket, struct Result *result)
{
    int total = 0;
    int bytes;

    while (total < sizeof(struct Result))
    {
        bytes = send(socket,
                     (char *)result + total,
                     sizeof(struct Result) - total,
                     0);

        if (bytes <= 0)
            return -1;

        total += bytes;
    }

    return 0;
}

int main()
{
    int serverSocket, clientSocket;

```

```
struct sockaddr_in serverAddr;

int n;
int matrix[MAX][MAX];

int i, j;

int diagonal = 1;
int upper = 1;
int lower = 1;

struct Result result;

/* Create Socket */

serverSocket = socket(AF_INET, SOCK_STREAM, 0);

if (serverSocket < 0)
{
    perror("Socket Creation Failed");
    exit(1);
}

printf("Socket Created Successfully...\n");

/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);

/* Bind */
```

```
if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) < 0)
{
    perror("Bind Failed");
    close(serverSocket);
    exit(1);
}

printf("Bind Successful...\n");

/* Listen */

if (listen(serverSocket, 5) < 0)
{
    perror("Listen Failed");
    close(serverSocket);
    exit(1);
}

printf("Waiting for Client...\n");

/* Accept */

clientSocket = accept(serverSocket, NULL, NULL);

if (clientSocket < 0)
{
    perror("Accept Failed");
    close(serverSocket);
    exit(1);
}
```

```

}

printf("Client Connected...\n");

/* Receive N */

if (receive_int(clientSocket, &n) < 0)
{
    perror("Receiving N Failed");
    close(clientSocket);
    close(serverSocket);
    exit(1);
}

printf("Matrix Order Received : %d\n", n);

if (n <= 0 || n > MAX)
{
    printf("Invalid Matrix Order\n");

    close(clientSocket);
    close(serverSocket);

    return 1;
}

/* Receive Matrix */

printf("Received Matrix:\n");

for (i = 0; i < n; i++)
{

```

```

    for (j = 0; j < n; j++)
    {
        if (receive_int(clientSocket, &matrix[i][j]) < 0)
        {
            perror("Receiving Matrix Failed");
            close(clientSocket);
            close(serverSocket);
            exit(1);
        }

        printf("%d ", matrix[i][j]);
    }

    printf("\n");
}

/* SUM */

result.sum = 0;

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        result.sum += matrix[i][j];
    }
}

/* TRACE */

result.trace = 0;

```



```

for (i = 0; i < n; i++)
{
    result.trace += matrix[i][i];
}

/* MATRIX TYPE */
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        /* Diagonal check */

        if (i != j && matrix[i][j] != 0)
        {
            diagonal = 0;
        }

        /* Upper triangular check */

        if (i > j && matrix[i][j] != 0)
        {
            upper = 0;
        }

        /* Lower triangular check */

        if (i < j && matrix[i][j] != 0)
        {
            lower = 0;
        }
    }
}

```

```

/* Determine type */

if (diagonal)
{
    strcpy(result.type, "Diagonal Matrix");
}
else if (upper)
{
    strcpy(result.type, "Upper Triangular Matrix");
}
else if (lower)
{
    strcpy(result.type, "Lower Triangular Matrix");
}
else
{
    strcpy(result.type, "General Matrix");
}

/* Display calculated results */

printf("\n----- CALCULATED RESULTS -----\n");

printf("Matrix Type : %s\n", result.type);
printf("Sum          : %d\n", result.sum);
printf("Trace         : %d\n", result.trace);

/* Send results */

if (send_result(clientSocket, &result) < 0)
{

```

```

        perror("Sending Results Failed");
    }
    else
    {
        printf("Results Sent...\n");
    }

    /* Close */

    close(clientSocket);
    close(serverSocket);

    printf("Connection Closed...\n");

    return 0;
}

```

>>CLIENT

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define MAX 10

struct Result
{
    char type[100];
    int sum;
    int trace;
}

```

```
};
```

```
/* Send exactly one integer */
```

```
int send_int(int socket, int value)
{
    int total = 0;
    int bytes;

    while (total < sizeof(int))
    {
        bytes = send(socket,
                     (char *)&value + total,
                     sizeof(int) - total,
                     0);

        if (bytes <= 0)
            return -1;

        total += bytes;
    }

    return 0;
}
```

```
/* Receive complete result */
```

```
int receive_result(int socket, struct Result *result)
{
    int total = 0;
    int bytes;
```

```

while (total < sizeof(struct Result))
{
    bytes = recv(socket,
                  (char *)result + total,
                  sizeof(struct Result) - total,
                  0);

    if (bytes <= 0)
        return -1;

    total += bytes;
}

return 0;
}

```

```

int main()
{
    int sockfd;
    struct sockaddr_in serverAddr;

    int n;
    int matrix[MAX][MAX];

    int i, j;

    struct Result result;

    /* Create Socket */

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

```

```

if (sockfd < 0)
{
    perror("Socket Creation Failed");
    exit(1);
}

printf("Socket Created Successfully...\n");

/* Server Address */

serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);

inet_pton(AF_INET,
          "127.0.0.1",
          &serverAddr.sin_addr);

/* Connect */

if (connect(sockfd,
            (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) < 0)
{
    perror("Connection Failed");
    close(sockfd);
    exit(1);
}

printf("Connected to Server...\n");

/* Input N */

```

```
printf("Enter order of matrix : ");
scanf("%d", &n);

if (n <= 0 || n > MAX)
{
    printf("Invalid matrix order.\n");
    close(sockfd);
    return 1;
}

/* Input Matrix */

printf("Enter %d x %d matrix:\n", n, n);

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &matrix[i][j]);
    }
}

/* Send N */

if (send_int(sockfd, n) < 0)
{
    perror("Sending N Failed");
    close(sockfd);
    exit(1);
}

/* Send Matrix */
```

```

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        if (send_int(sockfd, matrix[i][j]) < 0)
        {
            perror("Sending Matrix Failed");
            close(sockfd);
            exit(1);
        }
    }
}

printf("Matrix Sent...\n");

/* Receive Results */

if (receive_result(sockfd, &result) < 0)
{
    perror("Receiving Results Failed");
    close(sockfd);
    exit(1);
}

/* Display Results */

printf("\n----- MATRIX RESULTS ----- \n");

printf("Matrix Type      : %s\n", result.type);
printf("Sum of Elements   : %d\n", result.sum);
printf("Trace of Matrix    : %d\n", result.trace);

```



```

        /* Close */

        close(sockfd);

        printf("Connection Closed...\n");

        return 0;
    }

```

OUTPUT

>>SERVER

```

Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Matrix Order Received : 3
Received Matrix:
1 0 0
1 2 0
1 2 3

----- CALCULATED RESULTS -----
Matrix Type : Lower Triangular Matrix
Sum          : 10
Trace        : 6
Results Sent...
Connection Closed...

```

>>CLIENT

```

Socket Created Successfully...
Connected to Server...
Enter order of matrix : 3
Enter 3 x 3 matrix:
1 0 0
1 2 0
1 2 3
Matrix Sent...

----- MATRIX RESULTS -----
Matrix Type      : Lower Triangular Matrix
Sum of Elements  : 10
Trace of Matrix  : 6
Connection Closed...

```